

System Architecture Masterclass

Reverse engineering didático do sistema real

Projeto: AI Support Assistant

Branch: main

Commit: 99f37860015b78a58134c18a63b6651a762780fe

Data: 06/09/2026

Método: leitura estática, evidência no código e validação contra documentação oficial

CONFIRMED = comprovado · STRONGLY INFERRED = inferido por múltiplas evidências · POSSIBLE = plausível · UNKNOWN = não determinável

Sumário

1. Resumo executivo e mental model
2. Mapa do monorepo e inventário executável
3. C4 e topologia de deployment
4. NestJS, React e composição frontend
5. Autenticação, autorização e trust boundaries
6. Filas, BullMQ, Redis Pub/Sub e semântica de entrega
7. PostgreSQL, modelo de dados e source of truth
8. LLM, prompt injection e guardrails
9. Fluxos reais: request, job, login e dado
10. Resiliência, falhas, observabilidade e escala
11. Decisões, riscos, guia de navegação e glossário

Como ler: cada afirmação sobre a implementação é classificada. Diagramas mostram somente relações sustentadas pelos arquivos analisados; alternativas AWS são analogias pedagógicas, nunca equivalências tecnológicas.

1. Resumo executivo

CONFIRMED. O sistema é uma aplicação full-stack modular com um API NestJS, duas interfaces React independentes por origem, um processo worker NestJS separado, Redis e duas instâncias PostgreSQL. O domínio é suporte ao cliente assistido por LLM: usuários autenticam, escolhem o contexto de empresa, consultam guidelines, enviam mensagens e recebem respostas assíncronas.

A classificação mais precisa é **arquitetura híbrida, modular, distribuída em processos**. Há uma fronteira real entre API e worker, comunicação assíncrona por BullMQ/Redis e comunicação em tempo real por Redis Pub/Sub + Socket.IO. Contudo, não há um conjunto de microservices autônomos com bancos por serviço: API e worker compartilham os dois bancos e o mesmo domínio. Portanto, "microservice" é uma descrição operacional do worker; o conjunto também tem características de **distributed monolith**.

Como pensar neste sistema: o navegador entra pelo app principal ou pelo Support MFE; a identidade é uma sessão opaca armazenada em Redis; o API valida essa sessão e o tenant ativo; operações rápidas usam REST; geração LLM e validação de guideline viram jobs; o worker lê PostgreSQL, chama Gemini/OpenAI, grava PostgreSQL e publica um evento efêmero para o API emitir no Socket.IO.

Achados de alto valor

- **Confirmado:** sessão própria em Redis, TTL padrão de 24h; não há JWT, OIDC, OAuth, SAML ou IdP externo no repositório.
- **Confirmado:** `chat-generate` e `guideline-validate`, com retry exponencial de 3 tentativas na geração.
- **Confirmado:** idempotência opcional por chave `(companyId,userId,key)` durante 24h, rate limit por janela fixa de 1 minuto.
- **Confirmado:** guidelines têm versões imutáveis, hash SHA-256 e última versão válida; o worker resolve a versão válida na execução.
- **Risco observado:** `chat:events` usa Redis Pub/Sub, cujo delivery é at-most-once; uma UI desconectada pode perder atualização, embora o estado durável permaneça em PostgreSQL.
- **Não comprovado:** circuit breaker, tracing distribuído, métricas Prometheus/CloudWatch, DLQ dedicada e Module Federation.

2. Mapa do monorepo

Caminho	Responsabilidade comprovada	Classificação
`apps/api`	NestJS HTTP API, Socket.IO Gateway, Prisma core/chat, autenticação, empresas, usuários, conversas e produtores BullMQ.	Container/processo
`apps/chat-worker`	Worker NestJS; consumidores de geração e validação; Gemini/OpenAI; publicação de eventos; ranking de modelos.	Worker/processo
`apps/web`	React/Vite app principal: login, shell, company switcher, users, companies, history e SSO handoff.	Frontend shell
`apps/support`	React/Vite support chat; Socket.IO enquanto há jobs pendentes; navegação SSO.	Frontend independente / MFE por origem
`apps/shared/auth`	Token storage, `apiFetch`, tipos de sessão, URLs de handoff e allowlist de origens.	Biblioteca compartilhada
`apps/api/prisma`	Schema core: User, Company, GuidelineVersion.	PostgreSQL `DATABASE_URL`
`apps/api/prisma-chat`	Schema chat: Customer, ChatMessage, Conversation.	PostgreSQL `CHAT_DATABASE_URL`
`docker-compose*.yaml`, `deploy/nginx`	Ambientes local/prod, redes, volumes, reverse proxy.	Deployment
`use-cases`, `docs`, `README.md`	Contratos de comportamento e documentação operacional; úteis, mas subordinados ao código quando divergem.	Evidence/support

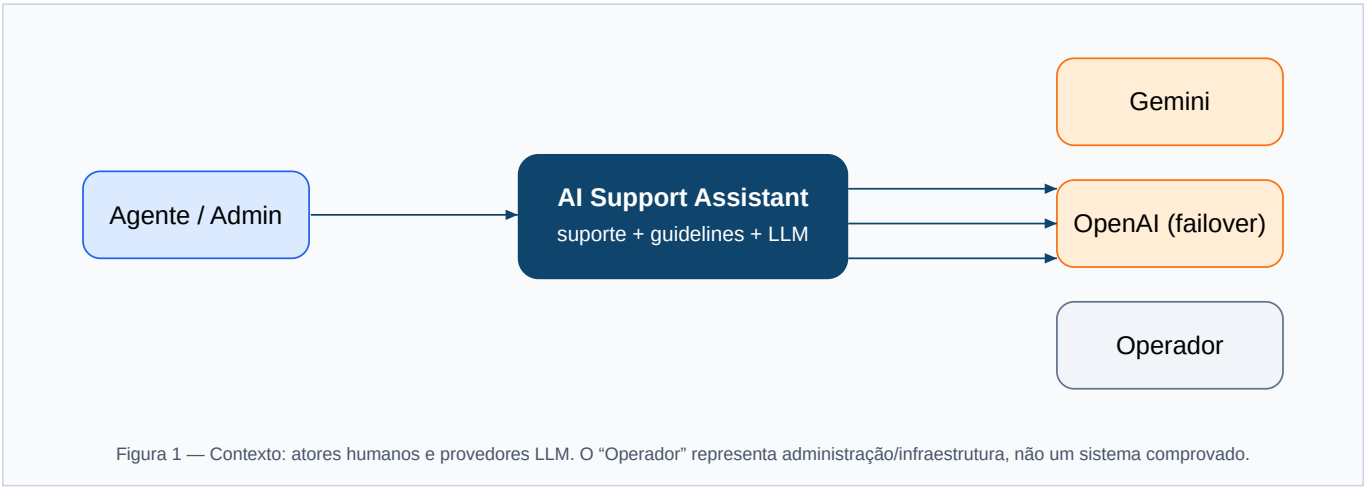
Evidência: README.md:1-72; docker-compose.yml:1-166; apps/api/src/app.module.ts:1-21; apps/chat-worker/src/app.module.ts:1-48.

Inventário executável

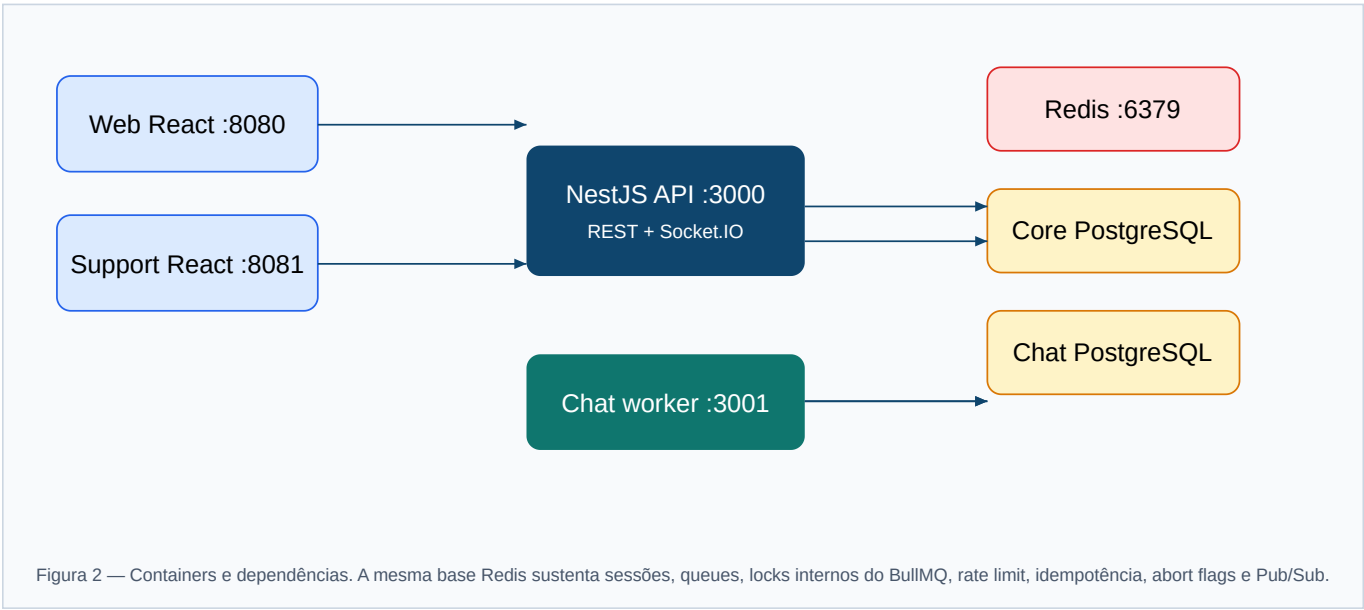
Nome	Entrypoint / porta	Dependências	Exposta / consome
API	`apps/api/src/main.ts`; 3000	core PG, chat PG, Redis	REST `/auth`, `/companies`, `/users`, `/chat`; Socket.IO
Chat worker	`apps/chat-worker/src/main.ts`; health 3001	Redis, dois PGs, Gemini; OpenAI opcional	consome 2 queues; chama APIs LLM; publica Redis
Web	Vite dev / nginx prod; 8080 local	API, Support origin	REST, SSO handoff
Support	Vite dev / nginx prod; 8081 local	API, Main origin	REST, Socket.IO

3. C4 e topologia

C4 Level 1 — System Context



C4 Level 2 — Containers (não significa Docker)



Analogia AWS: o API poderia ser executado atrás de ALB/ECS/EKS; o worker poderia escalar como serviço separado; Redis poderia ser ElastiCache; PostgreSQL poderia ser RDS. Isso é uma opção de deployment, não algo presente no repositório.

4. NestJS, React e padrões

NestJS: IoC e Dependency Injection

CONFIRMED. `AppModule` importa `AuthModule`, `ChatModule`, `CompaniesModule`, `PrismaModule`, `RedisModule` e `UsersModule`. Controllers recebem services no construtor; services recebem Prisma, Redis, ConfigService e filas por injection token.

```
constructor(
  private readonly prisma: PrismaService,
  private readonly redis: RedisService,
  @InjectQueue(CHAT_GENERATE_QUEUE)
  private readonly chatQueue: Queue<ChatGenerateJobData>,
) {}
```

Formalmente: **Inversion of Control** desloca a criação/ligação dos objetos para o framework; **Dependency Injection** fornece dependências prontas. Isso reduz acoplamento direto, facilita testes com doubles e centraliza lifecycle/configuração. O custo é uma dependency graph menos explícita em runtime e maior dependência das convenções Nest.

Evidência: apps/api/src/app.module.ts:10-20; apps/api/src/chat/chat.module.ts:12-31; apps/api/src/chat/chat.service.ts:52-65; apps/api/src/companies/companies.service.ts:67-75.

Padrões realmente observados

Padrão formal	Evidência e problema resolvido	Benefício / custo
Layered Architecture / Service Layer	Controller recebe HTTP e delega a `ChatService`, `CompaniesService`, `UsersService`; acesso a dados fica em Prisma services.	separa transporte e caso de uso; serviços podem acumular responsabilidades.
Repository-like Data Mapper	Prisma Client encapsula queries; não há classes Repository próprias generalizadas.	produtividade e tipagem; acopla domínio ao ORM.
Dependency Inversion / Adapter	`GeminiService` e `OpenAiService` oferecem operações equivalentes; worker seleciona provider.	failover e substituição; sem contrato TypeScript formal compartilhado.
Competing Consumers	Jobs BullMQ são consumidos por processadores worker; múltiplas instâncias poderiam competir pela mesma fila.	desacopla request de LLM; exige idempotência e observabilidade.
Publish/Subscribe	worker/API publicam em `chat:events`; `ChatGateway` subscrive e encaminha a rooms.	baixa latência e fan-out; perda de mensagens offline.
Idempotent Consumer / request idempotency	`SET ... NX` para chave de envio, condicionais `updateMany` por estado e `jobId` determinístico.	reduz duplicação; Redis vira dependência para replay e há janela de 24h.
Optimistic state transition	updates condicionados por `status` e checagem de `count`; evita sobrescrita após cancelamento/conclusão.	não bloqueia longamente; conflitos exigem tratamento.

Não comprovado: Clean Architecture, Hexagonal Architecture completa, CQRS, Saga, Event Sourcing e Domain-Driven Design como método adotado. Há conceitos compatíveis, mas não evidência suficiente para rotulá-los como arquitetura intencional.

React

Dois apps Vite usam React Context (`AuthProvider`), API client compartilhado e roteamento local por `App.tsx`. O Support abre Socket.IO apenas com jobs pendentes/processando. A composição é **Micro Frontend por origem e navegação/handoff**, não Module Federation: não aparecem `ModuleFederationPlugin`, `remoteEntry`, `exposes` ou `remotes`.

Por que um MFE? O código demonstra separação de deploy/origem e uma ponte de sessão; não demonstra independent deployment completo, versionamento de contratos ou isolamento de equipes. O ganho é separar shell/admin e chat; o custo é duplicar bootstrap/auth e lidar com sincronização entre origens.

Evidência: apps/web/src/SsoHandoffPage.tsx:1-56; apps/support/src/auth.tsx:35-84; apps/support/src/ChatPage.tsx:215-255; apps/shared/auth/index.ts:1-120; apps/web/vite.config.ts e apps/support/vite.config.ts não contêm Module Federation.

5. Identidade, autorização e segurança

O que SSO significa aqui?

CONFIRMED: há uma experiência SSO interna entre `web` e `support`, mas não um protocolo SSO federado. Login é email/senha contra core PostgreSQL; o API cria token aleatório de 32 bytes e guarda um JSON em Redis com `EX` de 86400 segundos. O Support recebe o token via fragmento `#token=`, armazena-o em seu próprio `localStorage` e chama `/auth/me`.



AuthN vs AuthZ

- **Authentication (AuthN):** `AuthService.login` compara bcrypt e cria sessão; `AuthGuard` exige `Authorization: Bearer` e procura a sessão Redis.
- **Authorization (AuthZ):** papéis `root`, `admin`, `manager`, `agent` e escopo de `activeCompanyId`; serviços rejeitam company IDs fora do contexto.
- **RBAC:** confirmado. Não há ABAC/ReBAC/ACL formal; atributos de tenant complementam RBAC.

Evidência: apps/api/src/auth/auth.service.ts:19-51; auth.guard.ts:16-40; session.service.ts:20-86; apps/api/src/users/users.service.ts:129-187; apps/api/src/companies/companies.service.ts:601-628.

OAuth/OIDC/PKCE/JWT/SAML

Conceito	Neste sistema	Lição
OAuth 2.0	N/A — não há Authorization Server/Resource Server/Grant.	framework de autorização, não sinônimo de login.
OIDC	N/A — não há ID Token, issuer, discovery ou UserInfo.	identity layer sobre OAuth.
PKCE/state/nonce	N/A como protocolo; há allowlist de return origins.	PKCE protege authorization code; allowlist reduz open redirect.
JWT/JWKS	N/A — token é opaco; não há assinatura/claims.	token assinado não é criptografado; aqui a confidencialidade fica no servidor Redis.
SAML	N/A.	alternativa enterprise baseada em assertions/ACS.

Guardrails observados: `ValidationPipe` com `whitelist` + `forbidNonWhitelisted`; DTOs e limite de arquivo; bcrypt; CORS configurável; allowlist de origins SSO; escopo server-side pelo Redis; soft delete; hash de guideline; checagem de ownership no worker; updates condicionais por estado. **Limitações:** valores de ambiente não foram expostos neste documento; não foi comprovado secret manager, rotação de credenciais, CSRF token, CSP, rate limit distribuído além de Redis, nem auditoria de login.

6. Filas, BullMQ e Redis

Inventário de filas

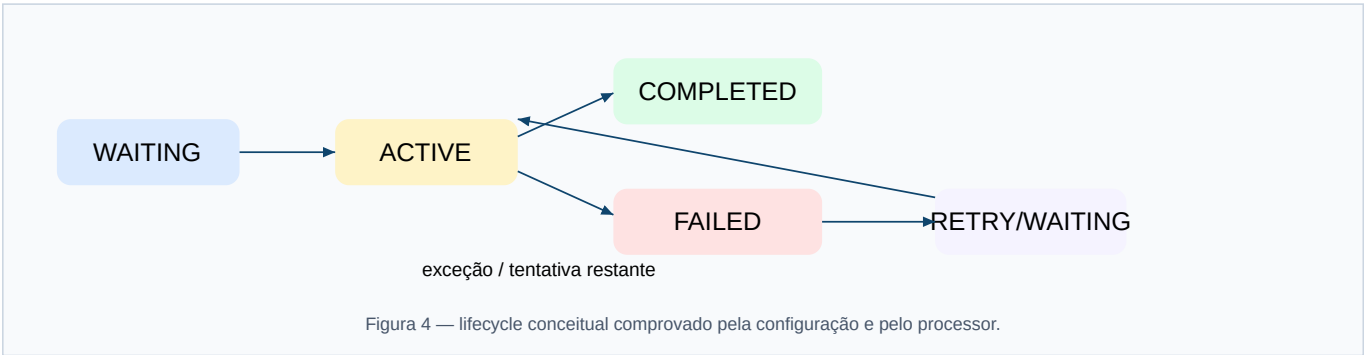
Queue	Job	Producer	Worker	Retry/backoff	Finalidade
`chat-generate`	`generate`	`ChatService.createUserMessage`, retry e failover	`ChatGenerateProcessor.process`	3 attempts default; exponential delay 1000ms; remove complete 100/fail 200	gerar resposta LLM e persistir
`guideline-validate`	payload `{companyId,versionId}`	`CompaniesService.enqueueValidationForVersion`	`GuidelineValidationProcessor.process`	configuração específica de retry não encontrada	classificar upload e ativar última versão válida

Evidência: apps/api/src/chat/chat.module.ts:15-27; apps/chat-worker/src/app.module.ts:21-35; apps/api/src/chat/chat.service.ts:183-198; apps/chat-worker/src/chat.processor.ts:27-50; guideline-validation.processor.ts:17-47.

Queue vs Pub/Sub vs Event Bus

Abstração	Topologia	Uso aqui	Garantia
Work Queue	producer → queue → consumer concorrente	geração/validação via BullMQ	estado persistido pelo BullMQ em Redis; retry de job
Publish/Subscribe	publisher → channel → subscribers ativos	`chat:events` → API Gateway → browser	Redis Pub/Sub at-most-once; sem replay
Event Bus	event → regras → targets	N/A; não há regras ou content-based routing	conceito AWS/EventBridge, não alegar equivalência

BullMQ: lifecycle e ACK



Onde está o ACK? Não existe um `ack()` explícito no código. A responsabilidade equivalente é o retorno normal de `process(job)` para sucesso e a exceção para falha; BullMQ coordena aquisição do job, lock/renewal e transição de estado. Isso não é o mesmo protocolo que SQS: em SQS o consumidor recebe, a mensagem fica invisível e o consumidor chama `DeleteMessage`; em BullMQ o worker conclui ou falha o job através do lifecycle interno.

Delivery semantic: para a fila, o comportamento operacional deve ser tratado como **at-least-once sob falhas**; duplicações podem ocorrer se o lock expirar ou houver crash em pontos ambíguos. O resultado de negócio é “effectively-once” apenas nos caminhos protegidos por estado condicional/idempotency; o LLM pode ser chamado novamente em retries. Redis Pub/Sub, por sua vez, é explicitamente at-most-once.

DLQ: ``removeOnFail: 200`` e o conjunto de jobs falhos não constituem automaticamente uma Dead-Letter Queue. O repositório não mostra uma fila dedicada, replay operator ou política de poison job. A UI oferece retry manual de mensagem falha.

7. Redis: papéis e source of truth

Papel	Quem grava/lê	TTL / perda	Source of Truth
Sessão opaca	Auth/SessionService; AuthGuard e MFES leem via API	TTL `SESSION_TTL_SECONDS`, default 86400; pode desaparecer	Redis para sessão; identidade cadastral é core PG
BullMQ backing store	API producers e worker consumers	retention 100 complete/200 fail para chat; perda pode impedir jobs pendentes	Redis para estado de fila; resultado de negócio em chat PG
Pub/Sub `chat:events`	worker/API publish; Gateway subscribe	efêmero, sem TTL/replay	não é source of truth; UI reconcilia lendo PG
Rate limit	Chat/Companies services `INCR` + `EXPIRE`	60s; reset/perda afrouxa proteção temporariamente	Redis contador efêmero
Idempotência	ChatService `SET NX`, resposta armazenada	24h; sem Redis, replay não pode ser decidido	Redis durante janela; mensagens persistem no PG
Abort flag	stop/cancel; worker consulta	30min; PostgreSQL `cancelled` reforça	PG status é durable; flag é coordenação rápida
Model ranking	ModelRankService grava/lê	refresh configurável, default 6h	config/fallback no código; Redis é cache de ranking

Redis ≠ PostgreSQL

Redis fornece estruturas rápidas e coordenação; PostgreSQL fornece durabilidade relacional, constraints e transações. O sistema deliberadamente mantém chat em uma base separada, mas sem FKs cross-database: `companyId`, `userId` e guideline IDs são soft references. Isso separa workloads e schemas, porém deixa consistência referencial no código.

Falha de Redis: login, autenticação de toda request, filas, Pub/Sub, rate limit e idempotência são afetados. Se Redis for limpo, sessões expiram logicamente e jobs BullMQ podem ser perdidos; dados duráveis de users/companies/guidelines/conversations/messages permanecem no PostgreSQL. Não há evidência de recuperação automática de sessões ou replay de Pub/Sub.

Redis Pub/Sub

O canal é um **Publish/Subscribe Pattern**: não é work queue e não é EventBridge. O publisher não conhece o consumidor; o Gateway recebe mensagens ativas e faz routing por tipo para room de usuário/empresa. Offline não recebe retrospectivamente. Isso serve para atualização de UI, pois a verdade pode ser relida via REST.

Evidência: apps/api/src/redis/redis.service.ts:15-29; apps/api/src/auth/session.service.ts:20-85; apps/api/src/chat/chat.gateway.ts:37-88; apps/api/src/chat/chat.service.ts:101-110,231-298; apps/chat-worker/src/model-rank.service.ts:24-89.

8. PostgreSQL e modelo de dados

Core database

Entidade	Por que existe	Constraints/indexes
Company	tenant e guideline atual denormalizada	PK cuid, `name @unique`, `currentGuidelineVersionId @unique`
GuidelineVersion	histórico imutável e auditável do texto enviado	PK; unique `(companyId,version)`; indexes por company/data/status; SHA-256, status e timestamps
User	identidade, bcrypt hash, papel e company default	PK; email unique; FK opcional Company

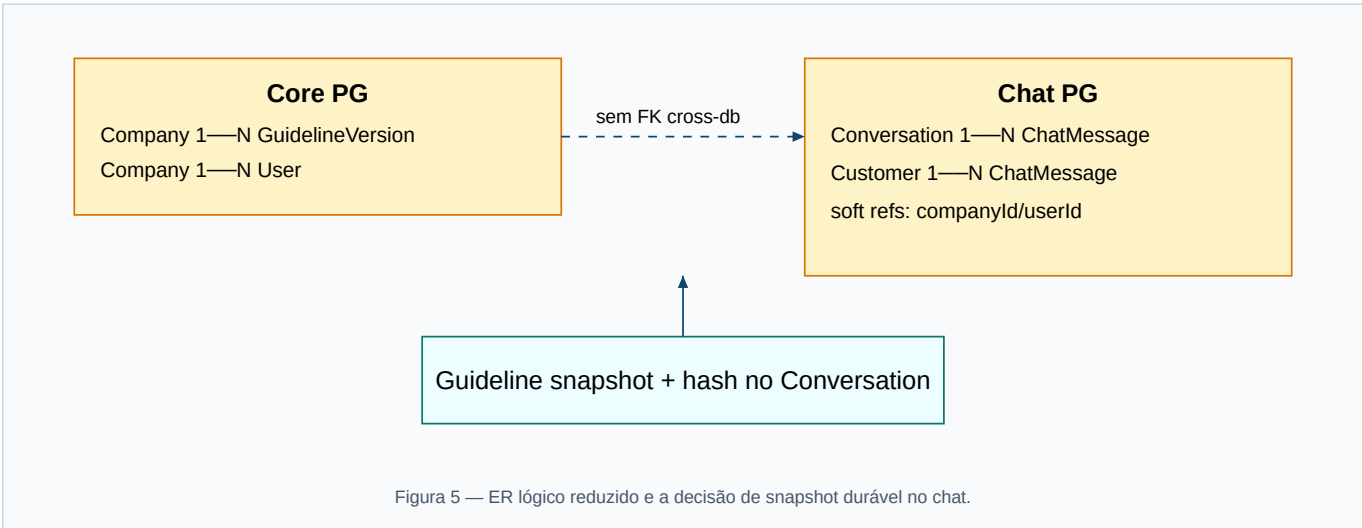
Chat database

Entidade	Relações e uso	Índices
Customer	cliente contextual por company/agent; sem FK cross-db	`companyId,createdById,updatedAt`
Conversation	thread, status final/reopen, rating, snapshot de guideline	`companyId,userId,lastMessageAt`; `companyId,status`
ChatMessage	mensagem user/assistant/agent; estado, retry, provider e auditoria de guideline	company/data, user/status, customer, conversation/data

Transações e locking

CONFIRMED: criação company + guideline usa `transaction`; criação de user+assistant e atualização da conversation usa transação; updates de user/remove lockam a company com `pg\_advisory\_xact\_lock(hashtext(companyId))`. Há updates condicionais por status no chat. Isso é locking pessimista/advisory em trechos de administração e optimistic conditional update em jobs.

Não confirmado: isolation level explícito, 2PC, distributed transaction ou Saga. Como há dois bancos, uma operação cross-database não é uma transação ACID única; o desenho usa separação de etapas e soft references. A criação de mensagem e placeholder é atômica dentro do chat DB; enqueue no Redis vem depois, com caminho de erro que marca failed.



9. LLM e segurança de prompt

CONFIRMED. Gemini é obrigatório no construtor (`^GEMINI_API_KEY``); OpenAI é opcional para failover. Há timeout de 30s, limite de 1.000 tokens de saída, ranking de modelos e prompt registry versionado em YAML.

Untrusted input / untrusted output

Guidelines e mensagens são conteúdo não confiável. O validador aplica allowlist implícita de conteúdo seguro por rejeição determinística de padrões de prompt injection, exfiltração, scripts, tracking e bypass de segurança; depois pode consultar um LLM em formato JSON. A regra importante é correta: o modelo é consultivo; uma guideline localmente perigosa não se torna válida pelo parecer do modelo.

Ameaça	Guardrail comprovado	Limitação
Direct/Indirect Prompt Injection	regexes para "ignore previous", exfiltração, scripts e bypass; guideline entra como contexto, não como system instruction.	regex não cobre todas as variações; não é prova de segurança semântica.
Excessive agency	não há ferramentas externas expostas ao LLM; worker só chama provider e DB via código.	não há action authorization/HITL para tool calls porque não há tool calling comprovado.
Output injection	texto vazio é rejeitado; placeholder application; UI exibe resposta.	sanitização/HTML encoding específico não foi demonstrado em todas as renderizações.
Cost/availability	1.000 tokens, timeout 30s, 3 attempts, rate limit 20/min e ranking top-N.	não há circuit breaker nem orçamento financeiro explícito.
Data leakage	prompt instruído a não revelar secrets/system prompt; guideline e mensagens são delimitadas.	instrução de prompt não é boundary determinística; não há DLP comprovado.

Prompt injection ≠ SQL injection: SQL injection explora linguagem interpretada pelo banco; prompt injection tenta influenciar um modelo probabilístico através de conteúdo contextual. A defesa contra prompt injection deve separar conteúdo não confiável, limitar ferramentas e impor autorização fora do modelo.

Evidência: `apps/chat-worker/src/guideline-validator.ts:20-99`; `gemini.service.ts:9-82`; `openai.service.ts:10-91`; `apps/chat-worker/prompts/registry.yml`; `apps/chat-worker/src/chat.processor.ts:158-232`.

Workflow de validação

Upload → GuidelineVersion ``pending`` → job ``guideline-validate`` → claim ``processing`` → screening determinístico → provider estruturado → transação que registra status e preserva última versão válida → Pub/Sub → Socket.IO company room. Se provider falha, status ``provider_error``; cancelamento possui estado próprio. Isso é um **asynchronous validation workflow**, não "AI security completa".

10. Comparações essenciais

SQS vs SNS vs EventBridge vs BullMQ

Conceito	SQS	SNS	EventBridge	BullMQ neste sistema
Abstração primária	Queue	Topic	Event Bus	Job Queue sobre Redis
Distribuição	um consumer processa; competing consumers	fan-out para subscriptions	rules → targets	um worker lógico por job; pode escalar horizontalmente
Filtragem/routing	não é content routing	filter policy em subscriptions	event patterns/rules	job name/data; roteamento implementado pela aplicação
Persistência	mensagem permanece até delete/retention	depende do subscriber; delivery via endpoints	serviço gerenciado com regras/targets	Redis state + retention configurada; não é banco de negócio
Retry/DLQ	visibility timeout + redrive policy/DLQ	políticas por subscription/endpoint	retry target + DLQ configuráveis	attempts/backoff; DLQ dedicada não encontrada
ACK	DeleteMessage	ack depende do protocolo subscriber	target delivery semantics	retorno/exception do processor; lock interno
Ordering	FIFO opcional	não assume ordering universal	não usar como fila ordenada	não há ordering configurado
Scheduling	delay/message timers	não primário	rules/schedules	delay usado no backoff; repeat não encontrado

Por que não SQS/SNS/EventBridge? O código é local/Compose e já depende de Redis para sessão, fila e eventos; BullMQ oferece o modelo de job necessário com baixa mudança. SQS seria atraente para durabilidade gerenciada e DLQ; SNS + SQS para fan-out durável; EventBridge para muitos produtores e content-based routing. Nenhuma dessas alternativas é “a mesma coisa” que Redis/BullMQ.

REST vs Queue

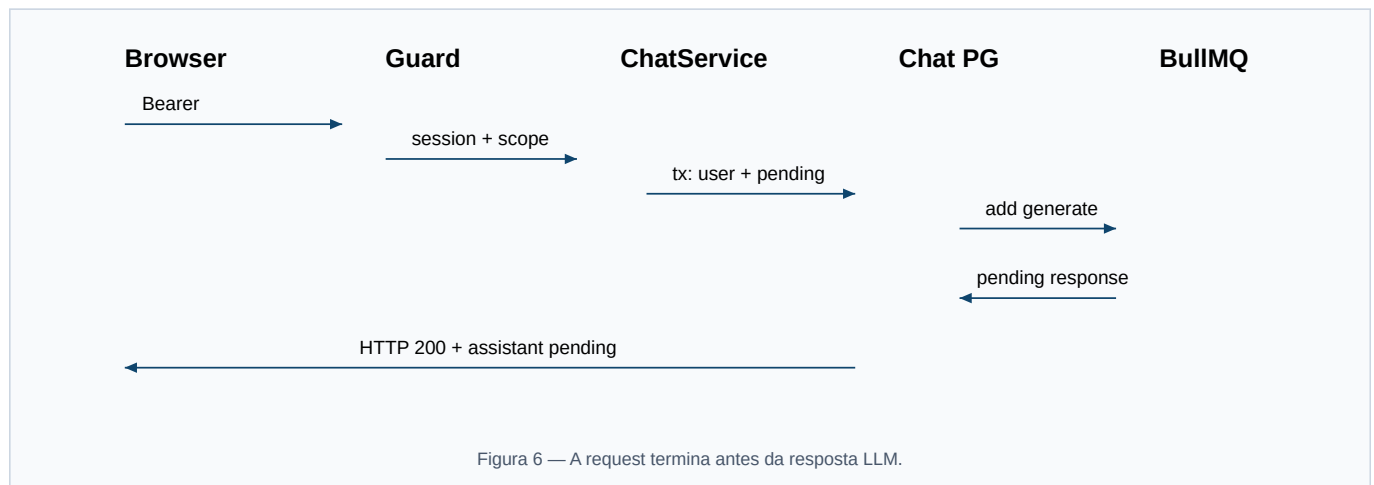
REST é síncrono: caller espera resposta, latência e falhas propagam. Queue é assíncrona: `/chat` persiste e responde `pending`, depois worker produz resultado. O desenho evita manter request HTTP aberta durante LLM, permite retry e escala independente; em troca introduz eventual consistency, estados intermediários e necessidade de reconciliação.

OAuth/OIDC/SAML; Session/JWT

O projeto escolhe sessão server-side opaca, adequada ao controle imediato por Redis e ao handoff entre apps. JWT reduziria lookup em cada request, mas complicaria revogação/rotação e não é observado. OIDC seria a escolha se houvesse IdP externo e múltiplas aplicações/organizações; não há essa infraestrutura no repositório.

11. Quatro vidas

A vida de uma request: POST `/chat`



1. Middleware não é registrado explicitamente; o Nest route pipeline entra no `AuthGuard`.
2. Guard lê Bearer, busca `session:token` no Redis e injeta `request.session`.
3. Service usa `activeCompanyId`, aplica idempotência e rate limit, resolve conversation.
4. Transação cria mensagem do usuário e placeholder assistant.
5. Queue recebe `generate` com `jobId` determinístico, attempts 3 e exponential backoff.
6. Resposta retorna `pending`; atualização posterior chega por Socket.IO.

A vida de um job

Criação → serialização no Redis pelo BullMQ → waiting → worker adquire/lock → `process` marca `processing` → lê contexto em dois PGs → checa abort → chama provider → update condicional `processing` → `completed` → publica evento. Exceção: failed/retry; Gemini esgotado com OpenAI configurado gera novo job de failover. Crash/lock stall pode permitir nova tentativa. **ACK**: retorno normal/exception do processor e lifecycle BullMQ, não chamada explícita.

A vida de um login

Web envia `/auth/login` → API consulta User e bcrypt → escolhe company ativa → gera 32 bytes hex → Redis `SET EX` → devolve token opaco. Support sem token redireciona ao `/sso/handoff`; Web valida `returnUrl` contra allowlist, passa `#token`; Support salva token por origin e faz `/auth/me`. Não há code, code_verifier, ID token, JWKS ou nonce.

A vida de um dado: guideline

Upload multipart → limite 10 MiB e extensão `.txt` → hash → Company/GuidelineVersion no core PG → versão `pending` → job de validação → status e `currentGuidelineVersionId` após validação → Conversation nova captura snapshot/hash → worker registra version/hash na mensagem. O source of truth da política é GuidelineVersion válida/core PG; o snapshot é source of truth da conversation histórica.

12. Workflows de negócio

WORKFLOW: enviar pergunta ao copilot

Objetivo: agente obter orientação contextual. **Atores:** agente, Web/Support, API, worker, Gemini/OpenAI, Redis, dois PGs. **Comando:** ``POST /chat`` significa “gere resposta”. O resultado futuro ``completed`` é evento/estado, não comando.

- Entrada: ``ChatPage`` faz POST; API autentica e escopa tenant.
- Persistência: mensagem user + assistant pending numa transaction.
- Assíncrono: BullMQ e worker, com 3 tentativas exponenciais.
- Consistência: resposta inicial é ``pending``; transcript final eventual.
- Segurança: ownership, status e abort são conferidos antes/depois da chamada LLM.
- Falha: 429 rate limit; 409 idempotency/final conversation; failed após attempts; retry manual; OpenAI failover opcional.

WORKFLOW: parar geração

``POST /chat/messages/:id/stop`` marca ``cancelled``, grava abort flag 30 min e remove queued jobs best effort. Job ativo pode não aceitar remove; o worker checa flag e status antes/depois da chamada e não grava completed. Isso é cancelamento cooperativo, não kill garantido do HTTP provider call.

WORKFLOW: substituir guideline

Manager/platform envia arquivo → version immutable → validação assíncrona → somente ``valid`` pode tornar-se ativa; versão anterior válida é preservada em falha. Conversations existentes mantêm snapshot, então a mudança não reescreve o passado.

WORKFLOW: resposta humana do admin

Platform admin pode criar mensagem ``agent`` na conversation; API publica ``agent_message``; Redis Pub/Sub → Gateway → room do owner. Isto é um “human-in-the-loop” de negócio, mas não uma aprovação de ação LLM.

WORKFLOW: company/user administration

Root/admin/manager operam dentro de regras RBAC; User update/remove usa transaction e advisory lock por company para impedir corrida na proteção do último root. Após remoção, sessões do usuário são varridas e deletadas em Redis.

Evidência: README.md:145-193; apps/api/src/chat/chat.service.ts:132-229,300-430; apps/api/src/users/users.service.ts:73-126,176-211; apps/api/src/companies/companies.service.ts:219-422; apps/api/src/chat/conversations.service.ts:523-634.

13. Falhas, resiliência e operação

Dependência	Falha observada/esperada	Recuperação comprovada	Unknown / risco
PostgreSQL core	login, roles, companies/guidelines indisponíveis; worker não resolve context.	Compose healthcheck/restart em prod.	backup/restore, replicas e failover não aparecem.
PostgreSQL chat	chat/history/worker writes falham.	erro de enqueue marca placeholder failed quando aplicável.	não há circuit breaker; retry de DB não demonstrado.
Redis	auth, queues, events, limits/idempotency falham.	ioredis retry settings; depend_on healthcheck.	single failure domain; persistence prod volume, RPO/RTO desconhecidos.
Worker	jobs acumulam; UI fica pending até reconciliação/timeout humano.	restart Compose; BullMQ pode reprocessar jobs stalled.	alerta de backlog/lag não comprovado.
LLM provider	timeout/erro, retry; Gemini esgota → OpenAI opcional.	exponential backoff, max attempts, manual retry.	sem circuit breaker/jitter explícito; retry storm possível.
Socket.IO/PubSub	update live perdido/offline.	UI faz REST/history e reabre socket quando pending.	evento efêmero; não há replay.
frontend remoto	Support/Web isolado indisponível; outra origin pode continuar.	handoff/redirect no bootstrap.	contrato de versão e fallback UX limitados.

Timeout, retry, backoff e jitter

Timeout de provider é confirmado: 30s. Retry é confirmado: attempts 3; backoff exponencial delay base 1s. Jitter não aparece (`backoff` não tem randomização), portanto não se deve chamar de exponential backoff with jitter. Circuit breaker também não aparece: retry sozinho não possui estados CLOSED/OPEN/HALF-OPEN. Rate limit é fixed window, via bucket temporal e `INCR`/`EXPIRE`; token bucket/sliding window não aparecem.

Deployment e escala

Compose prod cria containers API, worker, web, support, dois Postgres e Redis numa bridge `internal`; apenas web/support são publicados em loopback e API em 3000 loopback. Não há Kubernetes, Helm ou Terraform. Horizontal scaling do API é conceitualmente compatível porque sessão está em Redis; Socket.IO entre várias instâncias exigiria adapter Redis ou sticky strategy, não comprovada. Worker pode escalar como competing consumers, mas idempotência e LLM cost/backlog precisam ser monitorados.

Observabilidade

Há logs NestJS (`Logger`, `console.log`) e health endpoint do worker. Não foram encontrados métricas, OpenTelemetry, Prometheus, Grafana, Sentry, Datadog, CloudWatch ou correlation ID propagado API → job. Logo, os três pilares não estão completos: logs existem; métricas e traces são **UNKNOWN/NÃO COMPROVADOS**.

14. Architectural observations & risks

Características confirmadas

- Dois bancos separados por schema/workload, com soft references cross-db.
- Redis é um failure domain compartilhado entre auth, queue e realtime.
- Estado de negócio está em PostgreSQL; atualizações live são notificações.
- LLM é dependência externa com failover provider e limites operacionais.
- Tenant é derivado da sessão server-side, não do `companyId` enviado pelo cliente.

Riscos potenciais sustentados por evidência

- **Pub/Sub loss:** `ChatGateway` depende de mensagens ativas; Redis docs classificam Pub/Sub como at-most-once. Mitigação parcial: estado durável e polling/history.
- **Retry sem jitter:** configuração exponencial sem jitter, potencialmente correlacionando retries em falha de provider.
- **Distributed monolith:** API e worker compartilham schemas e regras; mudança de contrato exige coordenação.
- **Cross-db integrity:** soft refs não recebem FK; exclusões/renomes precisam ser governados pela aplicação.
- **Authentication concentration:** perda de Redis invalida todas as sessões; não há store secundário demonstrado.
- **LLM idempotency:** retries podem repetir chamada externa; persistência condicional evita dupla escrita, não custo/efeito externo.
- **Observability gap:** sem métricas/traces comprovados, backlog, p95, provider failure e reconexões são difíceis de operar.

Unknowns

- topologia real fora de Compose, backups, RTO/RPO, TLS termination e WAF;
- políticas de retenção/eviction do Redis em produção;
- provisionamento e rotação de `GEMINI\_API\_KEY`, `OPENAI\_API\_KEY` e credenciais PG;
- headers de segurança, CSP, sanitização final de markdown/HTML e auditoria formal;
- configuração de concorrência BullMQ e número de réplicas em runtime.

15. Perguntas de entrevista de System Design

1. Por que não chamar Gemini dentro da request? **Porque o desenho transforma latência variável em job, permite retry/failover e libera o caller; paga com eventual consistency.**
2. BullMQ é SQS? **Não. É uma job queue baseada em Redis; SQS é serviço gerenciado com visibility timeout/delete e DLQ.**
3. Onde está o ACK? **No lifecycle interno do worker: retorno resolve, exception falha; não há ack explícito.**
4. O sistema é exactly-once? **Não. O transporte/job deve ser tratado como at-least-once sob falha; há idempotência parcial e writes condicionais.**
5. Por que Pub/Sub e não fila para UI? **Atualização live é fan-out efêmero; o transcript durável é relido do PostgreSQL. Se perda fosse inaceitável, usar stream/queue.**
6. Qual é o source of truth? **Core PG para identidade/guidelines; chat PG para conversations/messages; Redis para sessão/coordenação efêmera.**
7. Como escalar workers? **Mais consumers na mesma queue, controlando concorrência, rate de provider, DB pool, custo e idempotência.**
8. Como proteger LLM? **Tratar input/output como não confiáveis, screen determinístico, limitar contexto/saída, autorizar ações fora do modelo; neste código não há tool calls.**

16. Glossário

Termo	Explicação
ACK / Acknowledgment	Confirmação de recebimento/processamento; BullMQ expressa isso pelo resultado/erro do processador.
At-least-once	Entrega/processamento pode repetir; exige Idempotent Consumer.
At-most-once	Entrega ocorre no máximo uma vez; pode perder, como Redis Pub/Sub.
Backoff	Atraso entre retries; aqui exponencial com base 1s, sem jitter comprovado.
Bounded Context	Fronteira semântica de domínio; chat e core têm schemas separados, mas não independência total.
Competing Consumers	Vários workers competem para executar itens de uma work queue.
DLQ	Dead-Letter Queue dedicada para mensagens que excederam política; não comprovada aqui.
Eventual Consistency	Leitura inicial pode mostrar pending; resultado chega depois.
Fan-out	Uma publicação distribuída a múltiplas subscriptions.
Idempotência	Repetir a mesma operação produz o mesmo efeito observável.
IdP / RP	Identity Provider / Relying Party; conceitos OIDC não implementados aqui.
JWT / JWKS	Token estruturado assinado / conjunto de chaves públicas; N/A neste sistema.
MFE	Micro Frontend; aqui composição por origins, não Module Federation.
PKCE	Proof Key for Code Exchange; proteção do Authorization Code Flow, N/A.
RBAC	Role-Based Access Control; roles e hierarquia observadas.
Source of Truth	Sistema/registro autoritativo para um dado.
TTL	Time To Live; expiração de sessão, flags, contadores e claims temporários.
Visibility Timeout	Semântica SQS: mensagem fica invisível após receive até delete/expiração; não é BullMQ ACK.

17. Referências

Source-code evidence

- `README.md` — arquitetura, endpoints, fluxos, ambientes e garantias declaradas.
- `apps/api/src/auth/*`, `apps/api/src/chat/*`, `apps/api/src/companies/*`, `apps/api/src/users/*`.
- `apps/chat-worker/src/*` — processors, providers, prompt registry e eventos.
- `apps/api/prisma/schema.prisma` e `apps/api/prisma-chat/schema.prisma`.
- `docker-compose.yml`, `docker-compose.prod.yml`, Dockerfiles e nginx configs.

Referências externas oficiais

- [BullMQ Documentation](#) — queues, workers, retries e stalled jobs.
- [Redis Pub/Sub](#) — delivery semantics at-most-once, channels e subscribers.
- [Redis Streams](#) — contraste para persistência/replay.
- [Amazon SQS Visibility Timeout](#).
- [AWS decision guide: SNS vs SQS vs EventBridge](#).
- [Amazon EventBridge User Guide](#).
- [RFC 7636 — PKCE](#).
- [OpenID Connect Core 1.0](#).
- [RFC 7519 — JSON Web Token](#).
- [OWASP Top 10 for LLM Applications](#).
- [NestJS Custom Providers / DI](#).
- [React documentation](#).

Pesquisa externa realizada em 06/09/2026. Quando a implementação e a documentação do projeto poderiam divergir, o código/configuração local prevaleceu. Este material é um estudo read-only; nenhum arquivo do repositório foi alterado.

18. Catálogo de diagramas em texto

Os diagramas SVG anteriores são a fonte visual no HTML. Esta versão textual mantém os mesmos relacionamentos no PDF compatível com ambientes sem renderizador SVG.

```
CONTEXT0
[Agente/Admin] → [AI Support Assistant] → [Gemini]
                                     ↳ [OpenAI: failover]

CONTAINERS
[Web :8080] ┌
[Support] ─┴─> [NestJS API :3000] → [Redis]
                                     ↳ [Core PostgreSQL]
                                     ↳ [Chat PostgreSQL]
                                     ↳ [Chat worker :3001]

QUEUE / EVENT
[API producer] → [BullMQ: chat-generate] → [Worker]
[Worker/API] → [Redis Pub/Sub: chat:events] → [Gateway] → [Socket.IO rooms]

AUTH / SSO INTERNO
[Web login] → [API + bcrypt] → [Redis session:token]
[Web] — #token → [Support localStorage] — Bearer → [API AuthGuard]

JOB STATE
WAITING → ACTIVE → COMPLETED
          ↳ FAILED — retry/backoff → WAITING
          ↳ Gemini exhausted → OpenAI failover job

TRUST BOUNDARIES
Internet/Browser | API boundary | Internal Redis/PG | External LLM provider
untrusted       | AuthN/AuthZ  | durable/coordination | network + secrets
```

Figura 7 — Catálogo textual para impressão: contexto, containers, queues, SSO, lifecycle e trust boundaries.